

CSE5IDP Week 9 — Effective Reporting and the Final Handover

Lecture summary · Scott Mann · Semester 2, 2026

Milestone 3, Topic 5 — plus a preview of the DevSecOps CI/CD workshop

At a glance

The session ran about 1 h 45 min, with a short break at around 3:22 pm. Online attendees were Elsin, Firoz, Nisha and Jaya. This lecture closes the theory part of the subject; from here on it is handover and delivery. The content fell into four parts:

1. **The three handover artefacts** — a GitHub repository with a proper README, a **System Maintenance Document (SMD)**, and a **User Document**.
2. **How to structure the SMD**, with user stories as the thread that ties it together, and exactly what separates an A from a C in the design section.
3. **The usability trial** next Thursday on campus, and why its results earn you an extra SMD chapter.
4. **A preview of Scott's DevSecOps CI/CD workshop** (GitHub Actions → tests → security scans → container → Render.com), to be walked through in full next week.

"If you treat and develop user stories well, simple fact, you'll get a great grade for your SMD."

Housekeeping

- **Trade show grades** are being released now. The grade goes to whoever uploaded the submission and is distributed to the team. Scott aimed to finish before the next day's lab.
- **Tender grades** are expected around Monday or Tuesday of the following week.
- **Sample documents** (a research-focused SMD, a research report, a software SMD and a user doc) will be linked on the LMS under *Effective reporting*.
- **Public holiday Friday next week** (football finals), so there is no class that day. Scott will explain the lab plan in the next lab.
- **Scrum Master presentations** can be rescheduled flexibly (in labs, or via an MS Teams call) as long as they are **finished before the end of semester**. Scott prefers doing them face to face.
- The previous week's lecture recording had poor audio; Scott stayed close to the microphone this time.

1. The three handover artefacts

Artefact	Audience	Purpose
GitHub repository + README	Developers	Understand, install and run the project quickly
System Maintenance Document	Maintainers, the next development team	How the system was designed and built, and how to keep it running and extend it
User Document	End users	How to use the system as quickly as possible

The repositories are handed over as **private** repositories at the end of semester.

The README

Scott showed a Markdown README from a previous semester (a facility app). The baseline contents are:

- **Name and overview** — no more than a paragraph: what it does, what it is for, who uses it.
- **Quick start** — the shortest route to running it, ideally one or two commands (the example was `git clone`, `npm install`, `npx expo start`).
- **Implemented features.**
- **Project structure** — the directory layout and hierarchy.
- **Key components** and the **tech stack**.
- **Contributing statement** — not needed for IDP, since most projects are not open source.
- **Licence.**

Choosing a licence

Use a licence chooser site and answer its questions. Scott said **MIT or GPL** are generally appropriate. Project owners are not licensing experts, so **the licence you attach will probably be the one they release under**. Choose what you think fits best and document your reasoning for the owner. For a university research project that others might extend, where contributions should flow back and the original must be attributed, he suggested MIT or GPL.

Showing your code to third parties (for example, in a portfolio) needs the project owner's express permission, because the work is private between the team and the owner. Summaries, abstractions and selected snippets are usually fine. Owners know students use these projects as portfolio pieces, so have the conversation and find out where the line sits.

2. The System Maintenance Document

Purpose and flexibility

The SMD gives anyone maintaining the solution the knowledge of **how you designed it and how you built it**. Projects in the subject fall into two broad kinds, and the rubric is interpreted to suit:

- **Software projects** (web or mobile apps and similar), where the SMD documents the sys-

tem.

- **Research projects** (for example the LLM bias-evaluation project and Leafy AI), where the SMD lays out the **experimental protocol**: how findings are derived and interpreted, and how to extend the experiment. The research report itself is the main deliverable, and the SMD supports it.

Hybrid projects (some code, some report) are fine too.

What goes in a research-oriented SMD

Based on the Semester 2, 2025 LLM bias sample:

- Core modules, with a one-line summary of what each script does.
- **Explicit dependencies.**
- **Domain knowledge a generic developer would not have** — formulas and specialised concepts, either in the main body or in an appendix.
- **Maintenance guidelines** — which scripts to run, when to insert data, and how outputs flow through the pipeline. Describe any point where a user must actively intervene.
- **User interface rationale** (if there is one) — how it is composed and *why*.
- **Tech stack and architecture** — front end, logic layer, database, and the schema (ER diagrams and data contracts).
- **Software versions and dependencies** — beyond `requirements.txt`, give guidance on how to **audit and keep dependencies up to date**.
- **Directory layout, installation and start-up**, how to run the system from different perspectives, and prompt guidance for LLM work.

Data refresh and scale. A student asked how to document future data needs. Scott's answer: document what is appropriate now (data size, refresh frequency), then add **future-extension recommendations**. These are not bound by current constraints, and they hand the resourcing decision back to the project owner.

Security documentation. Elsin asked whether a cybersecurity team's three extra documents (system administration; backup, restore and disaster recovery; securing the system against the OWASP Top 10) were required. Scott said yes: all three are core maintenance knowledge.

Lift-and-shift tip: write the SMD's project description, package structure and architecture sections so they can be pasted straight into the repository README.

The software SMD sample (a La Trobe parking app)

Scott stressed **not to panic about page counts**. The sample ran to 82 pages, but most of that is diagrams, tables, appendices and large screenshots, not dense text. Feel free to adapt its table of contents. There is **no length limit** on sections, but Scott's team marks more than 60 of these documents, so **be concise**: irrelevant or overly verbose material is a distraction.

Its structure, in order:

1. **Introduction** — what the system is, the document's audience, purpose, conventions and a change log.
2. **Design scope** — what the system does and the value it provides, including for different stakeholders (in the sample, a regular user versus an admin).
3. **Design constraints** — the hard limits the project owner set (for example, a mandated stack

or data structure).

4. **Reference documents**, in three groups:

- *existing software documentation* (prior teams' handovers, if you are extending a project);
- *further system documentation*;
- *vendor documentation* (every package, tool and API you built on, and where it came from).

Full academic referencing is optional; it is appropriate mainly for research projects.

5. **User stories** — used across three or four chapters (see below).

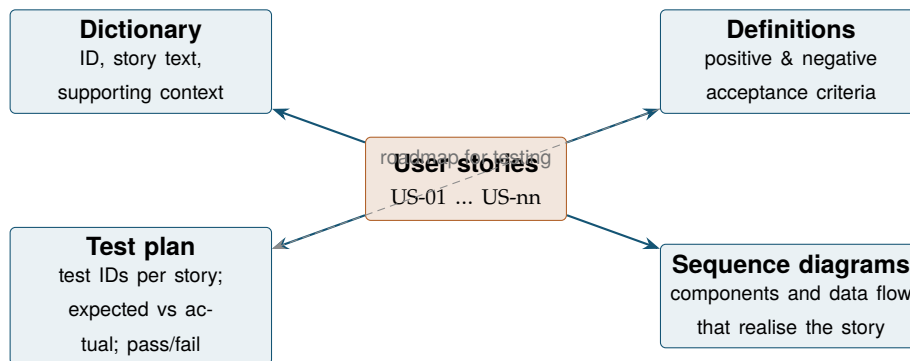
6. **Wireframes and design**.

7. **Technical design** — architecture, domain and class model, schema, sequence diagrams.

8. **Usability results, test plan**, and appendices (index, glossary).

User stories are the thread through the SMD

User stories follow the usual form: *as a (user), I want (goal) so that (outcome)*. They are derived from the requirements document you were given at the start of semester. The same **story IDs** are reused in each of four places:



One set of story IDs threads through four SMD chapters.

1. **Dictionary** — a plain definition of every story with an ID and some context. Having three or four stories is fine; so is having thirty.
2. **Definitions with acceptance criteria** — restate each story and list its **positive** criteria (what happens when things work) and **negative** criteria (what happens when a third-party API fails, or a value goes past a threshold). Story points are optional.
3. **Test plan** — break each story into sub-test IDs, with the criterion, expected output, actual output and a pass/fail result.
4. **Sequence diagrams** — one per key story, showing how components exchange data over time to make that story happen. For a scientific workflow, show the calls and data types passed between stages.

Be authentic about failures. Do not mark everything as passing when it is not. Failing tests **will not lower your mark**. They show the next team and the owner where to start and where to put resources. “Don’t let us discover it — be transparent.” No 12-week project gets everything done.

Scott’s diagram preferences:

- **Skip Venn diagrams** of the user stories, even though they appear in older samples.
- **Sequence diagrams are preferred.**
- **The object dictionary is optional.** “If section 5.5 were to disappear from your SMDs, I wouldn’t lose any sleep over it.”

Wireframes: A versus C

Scott showed the sample's wireframe section and said he would give it a **C**, possibly a **D**. It was a visual dump: no annotation, no captions, no context, so the marker has to guess at the design thinking. For an **A**:

- show the **evolution** of the design (for example, the state at week 4, week 8 and week 12);
- **annotate on top of the screens** with arrows, circles, highlights or call-outs that explain *why* the navigation, layout and grouping are the way they are;
- give every figure a caption with explanatory text.

"I've just told you how to get an A in your design section. Anything but an A is gonna disappoint me."

Technical design

Element	Scott's view
High-level system architecture diagram	Non-negotiable for every project. For a research project this is a pipeline diagram (data → cleaning → models → collation → interpretation → visualisation); for software it is the tiers, APIs and database.
Domain model / class diagram	Include it if the project has that structure.
Database schema	Show it.
Sequence diagrams per user story	Very important.
Object dictionary	Low value; optional.

The parking-app architecture diagram was the good example. From one picture you could infer the whole system: a mesh of sensor nodes around campus, reporting car-park availability to a central database, which a mobile front end reads.

3. The usability trial

When and where: Thursday next week from 10:00 am, outside the library (the Agora). There will be tables, monitors, banners and free snacks and drinks. Teams book **half-hour slots** on a spreadsheet Scott will post. It is **optional, but good teams do it**. Online students should run an equivalent trial with friends, family or colleagues; guidance will be posted on the LMS.

Why it is worth doing:

- It gives you **an extra SMD chapter** and a chart for the final presentation.
- It shows the owner you built the system *and* sought feedback on it. "You want to win the project for your team, this is how you go about doing it."
- **Take photos** of your team with the university community for LinkedIn and for the second-to-last slide of your final presentation.

How to run it:

1. Ask each participant to complete two or three user stories on a laptop or tablet.

2. Collect responses with a short form (Google Forms, Microsoft Forms or SurveyMonkey) using a five-point Likert scale (“The app was very intuitive”: strongly disagree → strongly agree), plus a free-text field. It should take 20–30 seconds.
3. **Six participants is enough, ten at most.** You are looking for trends, not commercial-grade statistics; the sample had 33, which is more than needed.
4. In the SMD, chart the results without overdoing it, list what worked and what needs improvement, and put the questionnaire in an appendix.

As with tests, **report negative feedback openly**. Telling the owner where users struggled is the intelligence they need to decide what to fund next.

4. The User Document

The user doc serves **end users** and should get them productive as quickly as possible.

- **Research and LLM projects** without a general-public audience may **fold the user doc into a chapter of the SMD**.
- **Software projects** need a separate user doc.

Suggested contents:

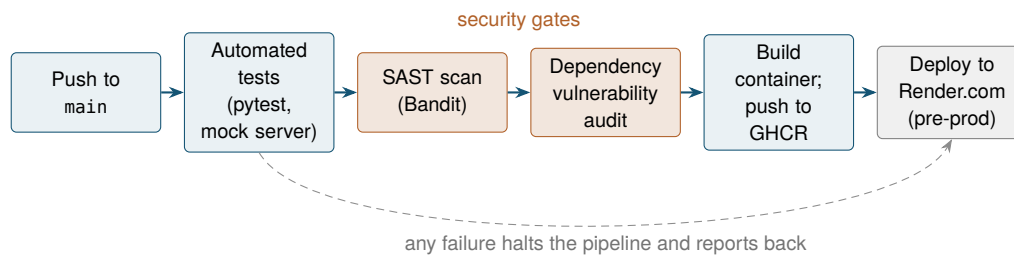
- **Introduction** — how to read and use the document.
- **Quick start** — prerequisites and how to interpret the interface.
- **User manual** — this is where most of your effort goes. Give a visual run-through for each user story: screenshots with guidance, and icons, bold text or colour to show what to click.
- **Administrator guide** — if there are distinct roles (the sample had a student role and an admin role).
- **Installation guide and hardware recommendations** — pitched at the owner’s level. Do not document general Linux or Windows usage unless the owner needs it. Flashing an SD card is usually assumed knowledge.
- **Glossary**, and a statement of authorship as an appendix.

Comic-strip layout. Scott dislikes full-page screenshots stacked one after another. Tile each story’s steps in rows and cells on **one page**, like a comic, so users never flip pages mid-task. For command-line tools, show terminal screenshots of the commands and of the warning, result and notification messages at runtime. If Android and iOS behave the same, **document one**; avoid duplication.

Use the samples to structure your thinking, but **don’t copy them**. Include what is relevant to your project.

5. Preview: DevSecOps CI/CD workshop

Scott spent the Tuesday building a demo that he will walk through in detail next week, with code and slides on the LMS. It is a Python Flask “online banking” app, chosen because it is high-risk and so worth getting right. It uses **GitHub Actions** as the CI/CD runner. Jenkins or Azure DevOps would also work, but the team is already on GitHub. A push to main triggers the whole chain, and any failing stage stops the pipeline.



Scott's demo pipeline, triggered by every commit to the main branch.

The demo deliberately failed at each stage:

- **Tests.** A dollar amount entered as an alphabetic string broke the code, so the pipeline halted until it was fixed.
- **Bandit (SAST).** Two medium-to-high findings:
 - Flask running with **debug mode enabled**, which exposes the debugger and could allow arbitrary code execution;
 - a hard-coded **0.0.0.0 binding** to all interfaces on the development server.

The fix was to turn debug off and bind to localhost.

- **Dependency audit.** Flask and pytest versions had known vulnerabilities. Bumping them in `requirements.txt` made the pipeline re-run and pass with no known vulnerabilities.
- **Deploy.** The container went to the GitHub Container Registry. Render.com was linked with an access token (Singapore, the closest free region), and every later commit redeploys automatically.

Two smaller notes: a macOS AirPlay receiver occupied port 5000, so the app ran on 5001. If the YAML is new to you, have Copilot or another AI explain it. Scott's pitch: few teams set this up because it looks like overhead, but those that do "have superpowers." It unites the cyber, software and IT students, and it is the norm in industry.

6. After-class discussion (another team)

The Leafy AI team raised computer-vision problems in the lab setup. Camera resolution is too low to see leaf damage or even fiducial markers, and lab staff keep moving the cameras, which shifts the regions of interest. Scott's advice applies to anyone who depends on a client's environment:

- **Document hardware and software limitations** explicitly.
- Write a technical statement for the client: *here is what we need for reproducibility, and here is what is making the analysis unreliable*. Then negotiate: "here's what you really shouldn't do, here's what we can tolerate."
- **Start general** (for example, overall green coverage or plant height) before chasing per-plant detail.

Action items

- ☐ Draft the README to the baseline contents; choose and document a licence.
- ☐ Build a user-story dictionary with IDs and positive and negative acceptance criteria.
- ☐ Reuse those IDs for the test plan (honest pass/fail) and for the sequence diagrams.
- ☐ Draw the high-level system architecture diagram (non-negotiable).
- ☐ Gather wireframes from each iteration and annotate them with design rationale.
- ☐ Book a usability-trial slot (Thursday next week, 10 am); prepare two or three tasks and a five-question Likert form. Online students: plan an equivalent remote trial.
- ☐ Check with the project owner what can be shown publicly as a portfolio piece.
- ☐ Get the Scrum Master presentation done before the end of semester.
- ☐ Next week: follow Scott's CI/CD workshop; consider adding the pipeline to the team repository.

Observations for the LJA project

- **The LJA is a hybrid project.** It is a Moodle-integrated tool *and* an evaluation of gap-detection accuracy. The SMD can use the research-style structure for the synthetic-data subsystem (generation protocol, GRM formulas in an appendix as “domain knowledge”, how to re-run the harness across seeds) alongside the software structure for the plugin and dashboard.
- **Licence.** Moodle is GPL-licensed and its plugins are expected to be GPL-compatible, so GPL (rather than MIT) is likely the natural choice for the plugin code. Record the reasoning for Scott.
- **Honest test reporting fits the metric story.** Reporting the 0.49–0.63 correlation *range*, and marking unmet criteria as failures, is exactly the transparency Scott asked for. The data-discipline rule (truth_* files never read by inference) belongs in the SMD's constraints and maintenance sections.
- **Architecture diagram.** Moodle extraction → semantic rubric-to-SILO mapping → competency clustering → gap detection → dashboard and export is the pipeline diagram, and each arrow is a candidate sequence diagram.
- **Usability trial.** The dashboard is a good fit for two or three trial tasks (for example “find your weakest competency”, “open a recommended resource”). Use synthetic students only; do not use real student data at a public stand.
- **DevSecOps.** Bandit plus a dependency audit in GitHub Actions would slot directly onto the Python synthetic-data stack and supports the subject's security-first mandate. The Docker Moodle dev environment's setup (port 8081, env.sh defaults) is ready-made SMD installation content, and the debug-mode lesson applies to Moodle's developer debugging settings before any demo.
- **README lift-and-shift.** Write the SMD overview, directory layout and quick start once and reuse them in the repository README.